ODFS vNext — Integration Map

Where each vNext module plugs into existing ODFS

Algorithmic specification + insert points — March 2026

Nguyen Quy Tung (Kevin T.N)

March 2026

Abstract

This document answers one question: **given the existing ODFS architecture (IPOD layers I→D→K→O→U, VDP dynamics, Identity Monitor, Genesis Seed, three-zone gate, TIEN), where exactly does each vNext mechanism insert?** It provides (i) a master insert-point table, (ii) concrete algorithm sketches at each insert point, and (iii) a revised completeness map showing vNext status. It does *not* rewrite the base architecture — it extends it.

Contents

1 ODFS Base Layer Recap (one paragraph) **2**

2 Master Insert-Point Table **2**

3 Algorithm Sketches at Each Insert Point **3**

3.1 Insert 1: GenomeSynthesizer in D-Long (D layer) . . . 3

3.2 Insert 2: FitnessWindow after each O decision . . . 3

3.3 Insert 3: CellCycle + DivisionTrigger at end of U layer . . . 4

3.4 Insert 4: ReplicationPartitioner + DamageInjector at division . . . 4

3.5 Insert 4b: AttentionSeeder — heritable VEG bias at division . . . 5

3.6 Insert 5: SelectiveForgetting in U layer (D-Long update) . . . 7

3.7 Insert 6: MicroMutationEngine at population level . . . 7

3.8 Insert 7: MergeDistillProtocol at population level . . . 8

3.9 Insert 8: SpecializationRouter in O layer . . . 8

4 Revised Runtime Engine (vNext pseudocode) **8**

5 vNext Completeness Map **9**

1 ODFS Base Layer Recap (one paragraph)

The existing runtime runs one **IPOD cycle** per request: **I** projects input to field vector R_0; **D** initialises $U = \mathrm{UOBV}(R_0, C)$; **K** runs VDP + VEG for up to `maxSteps`; **O** applies the three-zone gate (assimilate / quarantine / excrete); **U** runs the Identity Monitor (dual-gate A_t, N_t), Dream Cycle, and TIEN (Ω^* update via $\mathcal{L}(\theta)$). A single cell processes one context stream. D-Long stores anchor C, anti-anchor C^-, and recent history.

What is missing for vNext: the system has no *cell population*, no *genome/genotype split*, no *division or replication*, no *fitness window*, no *internal selection*, no *micro-mutation under plateau*, no *selective forgetting policy*, no *specialization routing*, and no *merge/distill* across cells. These are the vNext additions.

2 Master Insert-Point Table

Where each vNext module lives in ODFS

vNext module	Insert after	What changes
GenomeSynthesizer Γ	D-Long (D layer)	D-Long gains `genotype` + `phenotype_seed` fields; Γ serialises them
Genotype / Phenotype split	D-Long update (U layer)	Ω^*, ε_U, VEG weights = genotype (slow); C, C^-, history = phenotype (fast)
CellCycleScheduler Ψ	End of U layer	Per-cell age/phase tracker; gates Division-Trigger
DivisionTrigger	End of U layer	Checks age + Stab + Id window; spawns two child cells
ReplicationPartitioner	DivisionTrigger fires	Copies genotype to both; splits phenotype-/store
DamageInjector	ReplicationPartitioner	Applies mandatory loss (MemoryDrop / ParamJitter / ModuleLoss / IdentityScar)
AttentionSeeder η	After DamageInjector	Samples η_1, η_2; derives child VEG init bias; stores η in P_{seed} (heritable)
FitnessWindow W_t	After each O decision	Sliding window over Stab, Id, Surv, Eff signals
InternalSelectionScorer F	End of epoch (population)	Scores each cell; SelectPrune removes weakest
MicroMutationEngine	SelectPrune (if plateau)	Perturbs soft params when F flat for $\geq k$ epochs
SelectiveForgetting	D-Long update (U layer)	TTL + utility scoring on history; identity-protected entries exempt
SpecializationRouter	O layer (after zone decision)	Field-dominant cell routes to specialised sub-module
MergeDistillProtocol	End of epoch (population)	Two compatible cells merge/distill into one
HierarchyBuilder	End of epoch (population)	Groups cells into coordinator + worker tiers
(Body-required: skip for now)		
EmbodiedMembrane	Before I layer	Gating on live sensor stream
MaterialGrounding	I layer input	Invariant features fused with R_0
EmbodiedRepair	Health failure (any layer)	Action-based recovery in sim/hardware

3 Algorithm Sketches at Each Insert Point

3.1 Insert 1: GenomeSynthesizer in D-Long (D layer)

Insert point: D layer — D-Long extended with genotype/phenotype fields

Current D-Long stores: C, C^-, Σ^-, history ring buffer.
vNext adds: `genotype` G and `phenotype_seed` snapshot.

Definition 3.1 (Extended D-Long)**.**

$$\mathrm{DLong}_{\mathrm{vNext}} := \big(C,\ C^-,\ \Sigma^-,\ \mathrm{hist},\ \underbrace{G,\ P_{\mathrm{seed}}}_{new}\big).$$

G contains Ω^*, ε_U, VEG weights, TIEN hyper-params.
P_{seed} contains a compressed snapshot of C, C^-, and history summary.

Input: Cell state X_t (full UOBV + DLong)
Output: Genome $\Gamma(X_t)$
$G_t \leftarrow \{\Omega^*, \varepsilon_U, w_{\mathrm{VEG}}, \lambda_c, \mu_s, \eta_e, \gamma_p\}$
$P_{\mathrm{seed}} \leftarrow \{C, C^-, \mathrm{summary}(\mathrm{hist}[-20:])\}$
selftest $\leftarrow$ check $\{\Omega^*$ symmetric, $\varepsilon_U > 0$, $\|C\| > 0\}$
checksum $\leftarrow \mathrm{SHA256}(G_t \| P_{\mathrm{seed}})$
return $(G_t,\ P_{\mathrm{seed}},\ \mathrm{selftest},\ \mathrm{checksum})$

Algorithm 1: GenomeSynthesizer Γ — called at division or checkpoint

3.2 Insert 2: FitnessWindow after each O decision

Insert point: O layer — append signal to sliding window W

After every zone decision, one signal tuple is appended. W is a ring buffer of length $w = 30$ (tunable).

Definition 3.2 (Signal tuple)**.** *At step t:*

$$\sigma_t := (\mathrm{zone}_t,\ \rho_{U,t},\ S_{\mathrm{id},t},\ \mathrm{clamp}_t,\ \Delta\Gamma_t),$$

where $\mathrm{clamp}_t = \mathbf{1}[R_{\max}\ hit]$, $\Delta\Gamma_t = \Gamma_t - \Gamma_{t-1}$.

Definition 3.3 (Windowed fitness metrics)**.**

$$\mathrm{Stab}(W) := 1 - \frac{|\{t \in W : \mathrm{clamp}_t = 1\}|}{|W|}$$

$$\mathrm{Id}(W) := \frac{1}{|W|} \sum_{t \in W} S_{\mathrm{id},t}$$

$$\mathrm{Surv}(W) := \frac{|\{t \in W : \mathrm{zone}_t = \mathrm{assimilate}\}|}{|W|}$$

$$\mathrm{Eff}(W) := \mathrm{Surv}(W)\ /\ \big(1 + \mathbb{E}_W[\Delta\Gamma_t^+]\big)$$

$$F(W) := 0.30\,\mathrm{Stab}(W) + 0.30\,\mathrm{Id}(W) + 0.30\,\mathrm{Surv}(W) + 0.10\,\mathrm{Eff}(W).$$

Input: Window W, current ρ_U, S_{id}, zone, clamp flag, $\Delta\Gamma$
Output: Updated W, scalar metrics
Append σ_t to W; if $|W| > w$ drop oldest
Compute $(\mathrm{Stab}, \mathrm{Id}, \mathrm{Surv}, \mathrm{Eff}, F)$ from definitions above
return $(W,\ \mathrm{Stab},\ \mathrm{Id},\ \mathrm{Surv},\ \mathrm{Eff},\ F)$

Algorithm 2: UpdateFitnessWindow — called after O decision each step

3.3 Insert 3: CellCycle + DivisionTrigger at end of U layer

Insert point: U layer — after TIEN update, before returning

CellCycle runs every cycle. DivisionTrigger fires when conditions met.

Definition 3.4 (Cell-cycle state)**.** *Each cell carries* (ϕ, a, g) *where* $\phi \in \{\mathrm{G1}, \mathrm{S}, \mathrm{G2}, \mathrm{M}\}$*,* a *= age in cycles,* g *= growth proxy* $= \sum_{\tau \le t} F(W_\tau)$*.*

$$\mathrm{Divide}(X_t, W_t) = \mathbf{1}\big[a_t \ge T_{\min} \ \wedge \ \mathrm{Stab}(W_t) \ge 0.6 \ \wedge \ \mathrm{Id}(W_t) \ge 0.5\big].$$

Input: Cell X_t, window W_t, population $\mathbb{P}$
Output: Updated X_t or two children (X_1, X_2)
$a_{t+1} \leftarrow a_t + 1$
$g_{t+1} \leftarrow g_t + F(W_t)$
Advance phase ϕ by transition rules
if $\mathrm{Divide}(X_t, W_t)$ **then**
  $\Gamma \leftarrow \mathrm{GenomeSynthesizer}(X_t)$
  $(X_1, X_2) \leftarrow \mathrm{ReplicationPartitioner}(X_t, \Gamma)$
  $(X_1, X_2) \leftarrow \mathrm{DamageInjector}(X_1, X_2)$
  $\eta_{\mathrm{par}} \leftarrow X_t.P_{\mathrm{seed}}.\eta$ (`None` if root cell)
  $(X_1, X_2) \leftarrow \mathrm{AttentionSeeder}(X_1, X_2, \eta_{\mathrm{par}})$
  Replace X_t in $\mathbb{P}$ with (X_1, X_2)
  return (X_1, X_2)
end
return X_t

Algorithm 3: CellCycleUpdate + DivisionTrigger — end of U layer

3.4 Insert 4: ReplicationPartitioner + DamageInjector at division

Insert point: Division event — immediately after DivisionTrigger fires

Input: Parent X_t, genome Γ
Output: $(X^{(1)}, X^{(2)})$ before damage
Both children inherit G_t (genotype) identically from Γ
Split history: $X^{(1)}$ gets odd-indexed entries, $X^{(2)}$ gets even-indexed
Split residue store 50/50 by recency
Decay counters: corrections, excretes reset to 0 in both
Both inherit C, C^- from P_{seed}
return $(X^{(1)}, X^{(2)})$

Algorithm 4: ReplicationPartitioner

Input: $(X^{(1)}, X^{(2)})$, context stress level $\eta \in [0, 1]$
Output: $(\tilde{X}^{(1)}, \tilde{X}^{(2)})$
`// v0 MemoryDrop: always applied`
Truncate history of each child to 50% of parent history
`// v1 ParamJitter: applied when` $\eta > 0.3$
if $\eta > 0.3$ **then**
  Perturb Ω^* off-diagonal by $\mathcal{N}(0, 0.01)$ in each child
end
`// v3 IdentityScar: one child gets slight C- nudge`
$X^{(2)}.C^- \leftarrow 0.97 \cdot C^- + 0.03 \cdot \text{noise}$
return $(\tilde{X}^{(1)}, \tilde{X}^{(2)})$

Algorithm 5: DamageInjector — forces divergence (non-identical twins)

3.5 Insert 4b: AttentionSeeder — heritable VEG bias at division

AttentionSeeder: Division event — immediately after DamageInjector

AttentionSeeder samples two independent random seeds η_1, η_2 and derives an initial VEG attention bias for each child cell. The seed is stored in the child's P_{seed} and is *heritable*: grandchildren drift from parent seed rather than resampling from scratch. This creates **attention lineages** — traceable specialization trees.

Definition 3.5 (Attention seed)**.** *Let $\eta \in \mathbb{R}^6$ be an attention seed vector. At division two seeds are sampled:*

$$\eta_1,\ \eta_2 \sim \mathcal{N}(0,\ \sigma_\eta^2 \mathbf{I}_6), \quad \sigma_\eta = 0.3.$$

Each seed is converted to a VEG initial bias via temperature-τ softmax:

$$w_{\text{init}}^{\text{VEG}} = \text{softmax}(\eta \,/\, \tau), \quad \tau = 1.0.$$

This replaces the uniform VEG prior $[1/6, \ldots, 1/6]$ at cell birth.

Definition 3.6 (Heritable drift)**.** *When a child divides, grandchildren seeds are derived as:*

$$\eta' = (1 - \alpha_\eta)\, \eta_{\text{parent}} + \alpha_\eta\, \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}_6), \quad \alpha_\eta = 0.15.$$

$\alpha_\eta = 1.0$ *recovers full random resampling;* $\alpha_\eta = 0$ *yields clonal attention.*

Note: Why not MicroMutation?

MicroMutation perturbs Ω^* in response to plateau — acts on *genotype*, triggered by stagnation. AttentionSeeder acts on *phenotype* (VEG weights), triggered by *birth*. Same DNA, different epigenetic reading frame.

Input: Children $(X^{(1)}, X^{(2)})$; parent seed η_{par} or `None`
Output: $(X^{(1)}, X^{(2)})$ with VEG init bias + stored η
if $\eta_{\text{par}} \neq$ *None* **then**
  $\eta_1 \leftarrow (1-\alpha_\eta)\,\eta_{\text{par}} + \alpha_\eta\,\mathcal{N}(0, \mathbf{I})$
  $\eta_2 \leftarrow (1-\alpha_\eta)\,\eta_{\text{par}} + \alpha_\eta\,\mathcal{N}(0, \mathbf{I})$
else
  $\eta_1, \eta_2 \sim \mathcal{N}(0, \sigma_\eta^2\,\mathbf{I}_6)$ `// root cell: fully random`
end
$w^{(1)} \leftarrow \text{softmax}(\eta_1/\tau)$
$w^{(2)} \leftarrow \text{softmax}(\eta_2/\tau)$
$X^{(1)}.\text{veg_init} \leftarrow w^{(1)}$
$X^{(2)}.\text{veg_init} \leftarrow w^{(2)}$
$X^{(1)}.P_{\text{seed}}.\eta \leftarrow \eta_1$ `// store for next generation`
$X^{(2)}.P_{\text{seed}}.\eta \leftarrow \eta_2$
return $(X^{(1)}, X^{(2)})$

Algorithm 6: AttentionSeeder — called after DamageInjector at division

Listing 1: ipod/d/attention_seeder.py

```
import numpy as np

N_FIELDS  = 6
SIGMA_ETA = 0.3   # initial spread
ALPHA_ETA = 0.15  # heredity: 0=clonal, 1=fully random
TAU       = 1.0   # softmax temperature

def softmax_temp(x, tau=TAU):
    e = np.exp((x - x.max()) / tau)
    return e / e.sum()

def sample_seed(parent_eta=None):
    noise = np.random.randn(N_FIELDS)
    if parent_eta is None:
        return SIGMA_ETA * noise
    return (1 - ALPHA_ETA) * parent_eta + ALPHA_ETA * noise

def attention_seeder(child1, child2, parent_eta=None):
    eta1 = sample_seed(parent_eta)
    eta2 = sample_seed(parent_eta)
    child1['veg_init']        = softmax_temp(eta1)
    child2['veg_init']        = softmax_temp(eta2)
    child1['p_seed']['eta']   = eta1
    child2['p_seed']['eta']   = eta2
    return child1, child2

# Usage at division:
#   par_eta = parent['p_seed'].get('eta', None)
#   c1, c2  = attention_seeder(c1, c2, par_eta)
#
# In K-layer (VDP step 0):
#   veg = cell.get('veg_init', np.ones(6)/6)
#   # after step 1 VEG updates dynamically as normal
```

Note: Lineage tree

Since η is stored and derived from parent η, the full attention lineage is reconstructable across generations. Plotting w^{VEG} over generations reveals which fields each lineage

specialises in — Logic-dominant vs Language-dominant emerge naturally from random seeds without explicit role assignment.

3.6 Insert 5: SelectiveForgetting in U layer (D-Long update)

Insert point: U layer — inside D-Long update, after anchor refresh

Currently D-Long does: record(R), updateAnchor(). vNext adds forgetting pass.

Definition 3.7 (Memory utility score)**.** *For each entry r_τ in history:*

$$u(\tau) := \underbrace{\mathrm{sim}(r_\tau, C)}_{\textit{identity relevance}} + \underbrace{\mathrm{recency}(\tau)}_{\textit{TTL factor}} - \underbrace{\mathrm{redundancy}(r_\tau, \mathrm{hist})}_{\textit{dedup}}.$$

Entries with $u(\tau) < \theta_{\text{forget}}$ are dropped, unless *they are identity-protected (i.e.* $\mathrm{sim}(r_\tau, C) > 0.8$*).*

Input: History ring buffer hist, anchor C, threshold $\theta_f = 0.2$
Output: Pruned history
foreach *entry* $r_\tau \in$ hist **do**
  $u \leftarrow \mathrm{sim}(r_\tau, C) + \mathrm{recency}(\tau) - \mathrm{redundancy}(r_\tau)$
  if $u < \theta_f$ ***and*** $\mathrm{sim}(r_\tau, C) \leq 0.8$ **then**
    Remove r_τ from hist
  end
end
return *pruned hist*

Algorithm 7: SelectiveForgetting — appended to D-Long update

3.7 Insert 6: MicroMutationEngine at population level

Insert point: Population level — after SelectPrune, when plateau detected

Definition 3.8 (Plateau detection)**.** *A cell is in plateau if $|F(W_t) - F(W_{t-k})| < \epsilon_p = 0.02$ for $k \geq 5$ consecutive epochs.*

Input: Cell X, plateau flag, age a
Output: Mutated cell X'
$\mu \leftarrow \mu_0 \cdot \exp(-a/\tau_{\text{anneal}})$
`// Mutate Omega off-diagonal softly`
foreach (i, j) *with* $i \neq j$ **do**
  $\Omega_{ij} \leftarrow \Omega_{ij} + \mu \cdot \mathcal{N}(0, 1)$
end
Project Ω (symmetrise, clamp to $[0, 1]$, set diagonal to 1)
`// Nudge VEG weights`
$w_{\text{VEG}} \leftarrow \mathrm{softmax}(w_{\text{VEG}} + \mu \cdot \mathcal{N}(0, \mathbf{I}))$
return X'

Algorithm 8: MicroMutationEngine

3.8 Insert 7: MergeDistillProtocol at population level

Insert point: Population level — after selection, optional

Two cells are merge-compatible if $\mathrm{sim}(C_1, C_2) > 0.85$ and $\|G_1 - G_2\|_1 < \delta_{\text{merge}} = 0.3$.

Input: Two compatible cells $X^{(a)}, X^{(b)}$
Output: Merged cell X^*
$C^* \leftarrow (C^{(a)} + C^{(b)})/2$
$\Omega^* \leftarrow (G^{(a)}.\Omega + G^{(b)}.\Omega)/2$
Project Ω^* (symmetrise + spectral clamp)
Merge histories: union, deduplicate by sim > 0.95, keep top-60 by utility
$C^{-*} \leftarrow (C^{-(a)} + C^{-(b)})/2$
Run `selftest` on X^*; if fail $\rightarrow$ abort merge, keep $X^{(a)}$
return X^*

Algorithm 9: MergeDistillProtocol

3.9 Insert 8: SpecializationRouter in O layer

Insert point: O layer — after zone=assimilate, before emit

Field-dominant cell gets routed to a matching specialised sub-module.

Input: UOBV U, module registry $\mathcal{M}$
Output: Routing tag + optional sub-module activation
$d^* \leftarrow \arg\max_i R_i$ `// dominant field`
tag $\leftarrow \{0 : \text{Emotion},\ 1 : \text{Logic},\ 2 : \text{Reflection},\ 3 : \text{Visual},\ 4 : \text{Language},\ 5 : \text{Intuition}\}[d^*]$
if $\mathcal{M}$ *has expert for tag* **then**
  Activate expert module; pass U for domain-specific processing
end
return (tag, expert_output)

Algorithm 10: SpecializationRouter

4 Revised Runtime Engine (vNext pseudocode)

The base IPOD loop changes in three places: (1) FitnessWindow appended after O, (2) CellCycle called at end of U, (3) population-level ops wrap the per-cell loop.

Listing 2: runtime/engine_vnext.ts — diff from base

```
// -- Population wrapper (NEW) ------------------------------
for (const epoch of epochs) {
  for (const ctx of contexts) {
    for (const cell of population) {

      // -- Per-cell IPOD loop (base, unchanged) ------------
      while (step++ < params.maxSteps) {
        const in_t   = ctx.input(t)
        const [U, dec, id] = baseStep(cell, in_t)

        // NEW: update fitness window after O decision
        W = updateFitnessWindow(W, U, dec, id)

        if (dec.zone === 'assimilate') {
          // NEW: specialization routing before emit
          const tag = specializationRouter(U, moduleRegistry)
```

```
          return emit(U, dec, tag)
        }
        if (dec.zone === 'excrete') { handleExcrete(U); break }
        // quarantine: continue K loop
      }

      // -- U layer (base: TIEN + anchor update) -----------
      params = U_update(residues, params, replayBuffer)
      dLong.record(U.R)
      dLong.updateAnchor()

      // NEW: selective forgetting inside D-Long update
      dLong.hist = selectiveForgetting(dLong.hist, dLong.C)

      // NEW: cell cycle + division at end of U
      const divided = cellCycleUpdate(cell, W, population)
      if (divided) break  // cell replaced by two children
    }
  }

  // -- Population level (NEW, end of epoch) ---------------
  population = selectPrune(population)          // F(W) ranking
  population = mergeDistill(population)         // compatible pairs
  population = buildHierarchy(population)       // coordinator/worker
  population = microMutate(population)          // if plateau
}
```

5 vNext Completeness Map

Component	Base ODFS	vNext spec	Body-req?
VDP + Ω^* + VEG	✓done	unchanged	No
Three-zone gate	✓done	unchanged	No
Dual-gate identity (A+N)	✓done	unchanged	No
TIEN $\mathcal{L}(\theta)$	✓done	unchanged	No
Genesis Seed	✓done	unchanged	No
GenomeSynthesizer Γ	—	Insert 1 (D)	No
FitnessWindow W, F	—	Insert 2 (O)	No
CellCycle + DivisionTrigger	—	Insert 3 (U)	No
ReplicationPartitioner	—	Insert 4 (div)	No
DamageInjector	—	Insert 4 (div)	No
SelectiveForgetting	—	Insert 5 (U/D)	No
MicroMutationEngine	—	Insert 6 (pop)	No
MergeDistillProtocol	—	Insert 7 (pop)	No
SpecializationRouter	—	Insert 8 (O)	No
HierarchyBuilder	—	pop-level	No
EmbodiedMembrane	—	later	**Yes**
MaterialGrounding	—	later	**Yes**
EmbodiedRepair	—	later	**Yes**
Inside/Outside boundary	—	later	**Yes**

Note: Implementation order

Recommended sequence for Integrate-now modules:

1. FitnessWindow (trivial, needed by everything else)
2. GenomeSynthesizer (serialisation of existing params)

3. SelectiveForgetting (standalone, high value)
4. CellCycle + DivisionTrigger + Replication + Damage
5. MicroMutationEngine + SelectPrune
6. SpecializationRouter
7. MergeDistill + HierarchyBuilder

Warning

Body-required modules (EmbodiedMembrane, MaterialGrounding, EmbodiedRepair) must *not* be faked with passive statistical filters. Skip them entirely until a sensorimotor loop (sim or hardware) is available. The Integrate-now modules above give full vNext population dynamics without embodiment.